Effect & Tactical Skill UI 연출 개발 가이드라인

AI 에이전트 협업 및 Visual Cue 연출 구현 표준 가이드북 (서고 탐색 패턴 기반)

프로젝트: WildCard (UE 5.7) **문서 번호:** WC-GDL-2026-10-EFFECT-01 **작성일:** 2026-10-14

적용 대상: 게임플레이 프로그래머, 테크니컬 아티스트, 시스템 기획자, AI 에이전트

1. 개요 및 설계 원칙

본 가이드라인은 마법사 전술스킬 **'서고 탐색(Search Library)'**과 같이 **[플레이어 모달 입력(선택창)]**, **[서버 로직 검증 및 실행]**, **[광역 시각적 연출(UI 애니메이션/VFX)]**이 결합된 복합 기능을 신규 개발할 때 따라야 하는 표준 아키텍처와 구현 절차를 정의합니다.

핵심 3대 설계 원칙:

- **Zero-Touch Main HUD (바이너리 충돌 원천 차단):** 메인 HUD(`WBP_HUD.uasset`)에는 개별 연출 로직을 절대 직접 구현하지 않습니다. 메인 HUD는 GameplayTag에 따라 독립 위젯을 생성/표시하는 **이벤트 라우터(Router)** 역할만 수행합니다.
- **독립 연출 위젯 (1 연출 = 1 WBP):** 모든 연출 UI는 `UWildVisualEffectWidget` 을 상속받는 독립 위젯 블루프린트로 제작되며, 연출 종료 시 스스로 `FinishVisualEffect()` 를 호출하여 자가 소멸합니다.
- **단일 NetMulticast 이벤트 버스:** 스킬/이펙트마다 별도의 NetMulticast RPC를 추가하지 않고, `AInGameState::Multicast_BroadcastVisualCue` 단일 통로를 공유합니다.

2. 4단계 표준 라이프사이클 (4-Phase Lifecycle)

서고 탐색과 같은 복합 UI 연출 기능은 반드시 아래 4단계로 역할을 분리하여 구현합니다.

Phase 1: Local Input
로컬 모달 UI 활성화
`FCardSelectionContext`
플레이어 카드 선택

Phase 2: Server Execution
서버 RPC 전송 및 검증
`USearchLibraryEffect`
비즈니스 로직 수행

Phase 3: Cue Broadcast
`FWildVisualCueData` 패킹
InGameState의
`Multicast_BroadcastVisualCue`

Phase 4: Client Presentation
독립 연출 위젯 동적 스폰
`WBP_SearchLibraryEffect`
애니메이션 후 자가 소멸

라이프사이클 단계	실행 위치 및 핵심 API	역할 및 가이드라인
Phase 1: 로컬 입력 모달	Local Client `PC->RequestCardSelection(Context)`	특정 스킬 전용 함수를 만들지 말고, 범용 컨텍스트(`FCardSelectionContext`)를 전달하여 모달을 띄웁니다.
Phase 2: 서버 검증 및 실행	Dedicated Server `UEffectBase::ApplyEffect`	클라이언트가 보낸 선택 인덱스와 자원 소모를 서버 권한으로 검증하고 실제 게임 상태(덱, 패, HP)를 갱신합니다.
Phase 3: 시각적 큐 브로드캐스트	Dedicated Server `UEffectBase::BroadcastVisualCue`	결과 수치 및 시전자/대상 정보를 담은 `FWildVisualCueData` 를 구성하여 전체 클라이언트로 전파합니다.
Phase 4: 독립 연출 위젯 재생	Client HUD `UWildVisualEffectWidget`	메인 HUD의 맵에서 매칭된 독립 위젯을 생성하고 `PlayVisualEffect` 호출 → 애니메이션 종료 후

라이프사이클 단계	실행 위치 및 핵심 API	역할 및 가이드라인
		`FinishVisualEffect`로 자동 소멸.

3. 실전 C++ & Blueprint 구현 워크플로우

Step 1: GameplayTag 정의

`Config/DefaultGameplayTags.ini` 또는 에디터의 프로젝트 세팅에서 신규 시각 연출 태그를 추가합니다.

```
// 예시:
Visual.TacticalSkill.Mage.SearchLibrary
Visual.TacticalSkill.Rogue.PeekHand
Visual.Effect.Damage.Fire
```

Step 2: C++ Effect 또는 TacticalSkill 작성

서버에서 비즈니스 로직 완료 후 `BroadcastVisualCue`를 호출하도록 작성합니다.

```
bool USearchLibraryEffect::ApplyEffect(const FEffectContext& Context)
{
    // 1. 서버 비즈니스 로직 수행 (카드 덱 순서 조작 및 드로우)
    // ...

    // 2. 시각 연출 브로드캐스트
    FWildVisualCueData CueData;
    CueData.CueTag = FGameplayTag::RequestGameplayTag(TEXT("Visual.TacticalSkill.Mage.SearchLibrary"));
    CueData.Instigator = Context.SourceUnit;
    CueData.Target = Context.TargetUnit;
    CueData.Value = TopCards.Num();
    BroadcastVisualCue(Context, CueData);

    return true;
}
```

Step 3: DataTable 등록

`DT_TacticalSkillData` 또는 `DT_CardData`에 해당 스킬/카드를 등록하고, `VisualCueTag` 필드에 등록한 태그를 할당합니다.

Step 4: 독립 연출 위젯 블루프린트 제작

1. 콘텐츠 브라우저에서 `UWildVisualEffectWidget`을 부모로 하는 신규 위젯 블루프린트 생성 (예: `WBP_SearchLibraryEffect`).
2. UMG 디자이너에서 원하는 2D 연출(카드 플립, 빛나는 이펙트, 사운드 등)을 제작.
3. Event Graph에서 `Event PlayVisualEffect(CueData)`를 구현하여 애니메이션을 재생.
4. 애니메이션이 끝나면 **반드시** **`FinishVisualEffect()` 함수를 호출**하여 위젯을 자가 소멸시킵니다.

Step 5: HUD 위젯 매핑 등록

`WBP_HUD`의 Class Defaults → `Visual Effect Widget Map`에 `[Visual.TacticalSkill.Mage.SearchLibrary → WBP_SearchLibraryEffect]`를 등록합니다. (C++ 및 메인 그래프 수정 없음)

4. [핵심 1] AI 에이전트 사전 프롬프트 템플릿

AI 에이전트에게 새로운 연출 기능 구현을 지시할 때, **아래 블록을 그대로 복사하여 프롬프트 맨 앞에 첨부**하십시오.

```
[WILD_CARD_ARCHITECTURE_CONTEXT: Effect & UI Presentation Pipeline]
반드시 다음 아키텍처 원칙과 가이드라인을 준수하여 구현하십시오:

1. 아키텍처 표준 (Zero-Touch HUD & Event Bus):
   - WBP_HUD.uasset 또는 UWildHUDWidget 내부에 개별 이펙트의 연출 로직을 직접 하드코딩하지 마십시오.
   - 모든 UI 연출은 UWildVisualEffectWidget을 상속받는 독립 위젯으로 동작해야 합니다.
   - 서버에서 클라이언트로의 시각 연출 통지는 개별 RPC를 추가하지 말고 AInGameState::Multicast_BroadcastVisualCue를 통해서만 발송하십시오.
   - UEffectBase를 상속받은 C++ 이펙트에서는 UEffectBase::BroadcastVisualCue 헬퍼 함수를 사용하여 큐를 브로드캐스트하십시오.

2. 데이터 구조체 및 태그 규약:
   - FWildVisualCueData (Source/WildCard/InGame/Data/BattleData.h)
     * CueTag: FGameplayTag (예: Visual.TacticalSkill.Mage.SearchLibrary, Visual.Effect.Damage 등)
     * Instigator / Target: TWeakObjectPtr<AActor>
     * WorldLocation: FVector
     * Value: int32 (수치 또는 인덱스)
     * ContextPayload: FName
   - 태그는 계층형 Fallback을 지원하므로 구체적 태그(Visual.Effect.Damage.Fire)가 없을 경우 상위 태그(Visual.Effect.Damage) 위젯이 자동 선택됩니다.

3. 모달 입력과 연출의 분리 (서고 탐색 패턴):
   - 유저의 선택이 필요한 기능은 [Phase 1: Local Modal Input] -> [Phase 2: Server RPC & Effect] -> [Phase 3: Visual Cue Multicast] -> [Phase 4: UWildVisualEffectWidget 연출]의 4단계를 엄격히 준수하십시오.
   - 플레이어 컨트롤러에 스킬 전용 함수를 추가하지 말고, FCardSelectionContext와 PC->RequestCardSelection()을 재사용하십시오.

4. 컴파일 및 빌드 가드레일:
   - C++ 수정 후 반드시 WildCardEditor 타깃 컴파일을 검증하고, 오류 발생 시 자가 수정을 수행하십시오.
   - 엔진 소스(UE_5.7/Engine/)를 수정하지 마십시오.
```

5. [핵심 2] 작업자 요구사항 작성 템플릿 및 실전 예시

5.1 요구사항 작성 템플릿 (작업자 양식)

기획자나 프로그래머가 AI 에이전트에게 구현을 요청할 때 빈칸을 채워 작성하는 표준 템플릿입니다.

```
# [기능명] 신규 이펙트 및 UI 연출 구현 요청

### 1. 기본 정보
- 기능 유형: (카드 이펙트 / 전술스킬 / 타일 상호작용 중 선택)
- 명칭: [예: 화염구 / 적 패 훔쳐보기 / 서고 탐색]
- 시각 연출 태그: Visual.TacticalSkill.[Class].[SkillName] 또는 Visual.Effect.[EffectName]

### 2. Phase 1: 로컬 입력 모달 (선택창이 필요 없는 경우 "없음")
- 모달 유형: [예: 카드 선택 모달 (RequestCardSelection) / 타깃 지정 모달]
- 표시 조건 및 전달 컨텍스트: [예: 내 덱 상위 3장 표시, 1장 선택]

### 3. Phase 2: 서버 비즈니스 로직 (Dedicated Server)
- Effect 클래스명: [예: UFireballEffect / UPeekEnemyHandEffect]
- 실행 조건 및 검증: [예: 마나 3 소모 검증, 사거리 검증]
- 상태 변경 내용: [예: 대상 유닛 HP 5 감소, 상대방 손패 1장을 묘지로 이동]

### 4. Phase 3 & 4: Visual Cue 및 연출 위젯 (Presentation)
- FWildVisualCueData 페이로드:
  * Value: [예: 입힌 피해량 / 선택된 카드 ID]
  * Instigator & Target: [예: 시전자 유닛 -> 대상 유닛]
- 연출 위젯 명칭: WBP_VisualEffect_[기능명] (UWildVisualEffectWidget 상속)
- 연출 상세 내용: [예: 화면 중앙에 대상 카드가 나타난 후 불타며 사라지는 2D 애니메이션]
```

5.2 실전 작성 예시: "도적 전술스킬 [적 패 훔쳐보기(Peek Enemy Hand)]"

실전 작성 예시:

- **기능 명칭:** 적 패 훔쳐보기 (Peek Enemy Hand) / 태그: `Visual.TacticalSkill.Rogue.PeekHand`
- **Phase 1:** `PC->RequestCardSelection`을 재사용하여 상대방의 손패 카드 목록을 모달에 팝업. 플레이어가 버리게 할 카드 1장 선택.
- **Phase 2:** 신규 C++ `UPeekEnemyHandEffect::ApplyEffect`에서 기력 40 소모 검증 후, 선택된 상대 손패 카드를 묘지로 이동.
- **Phase 3:** `BroadcastVisualCue`로 태그 `Visual.TacticalSkill.Rogue.PeekHand` 및 버려진 CardID를 전체 브로드캐스트.
- **Phase 4:** `WBP_VisualEffect_PeekHand` 위젯이 생성되어 화면 중앙에서 카드가 단검에 베어지며 사라지는 UMG 애니메이션 재생 후 자가 소멸.

6. Git 협업 및 바이너리 충돌 방지 체크리스트

작업 완료 및 Git 커밋 전 반드시 아래 항목을 검진하십시오.

점검 항목	점검 내용	상태
WBP_HUD 무결성	`WBP_HUD.uasset`의 Event Graph나 내부 노드를 직접 수정하지 않았는가?	**필수 확인**

점검 항목	점검 내용	상태
독립 위젯 상속	신규 연출 위젯이 `UWildVisualEffectWidget`을 부모로 상속 받았는가?	**필수 확인**
위젯 자가 소멸	애니메이션 종료 시점에 `FinishVisualEffect()`가 누락 없이 호출되는가?	**필수 확인**
단일 RPC 준수	새로운 NetMulticast를 선언하지 않고 `Multicast_BroadcastVisualCue`를 사용했는가?	**필수 확인**
GameplayTag 등록	사용한 태그가 `DefaultGameplayTags.ini`에 정상 등록되었는가?	**필수 확인**
컴파일 검증	`WildCardEditor` Win64 Development 빌드가 오류 없이 통과하는가?	**필수 확인**